Appendix B Exercises

Declare section bases, counters, alignment, instruction widths, relocation rules, and PC meanings. Expand pseudo-instructions before sizing.

GB-Q01 Two-pass labels

Text starts at `0x00400000`; data at `0x10010000`. Each listed instruction occupies four bytes.

```
.text
main:  addiu $t0,$zero,0
loop:  addiu $t0,$t0,1
       beq   $t0,$a0,loop
       nop
done:  jr    $ra
       nop
func:  addiu $v0,$zero,7
       jr    $ra
       nop
.data
.align 2
table: .word 0xdeadbeef
count: .word 3
str:   .asciiz "Hi"
```

1. Give all seven label addresses.
2. Give the branch address and encoded immediate.
3. Find target fields for hypothetical `j done` and `jal func` without inserting them into the listing or moving labels.
4. Give the last allocated data-byte address.

GB-Q02 Relocations

Object A's text begins at `0x00401000`. At offset `0x10` is `jal external`, whose resolved target is `0x00402000`. Separately construct `0x12345678` using `lui/ori`. ORI zero-extends the low half.

1. Find the final call target field.
2. Give the full word for opcode 3.
3. Give both constant halves and derive the high adjustment needed for `lui/addiu` instead.
4. Explain the encoded placeholder and relocation metadata's different roles.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Exercises (continued)

GB-Q03 Merging sections

P contains 192 text bytes and sixteen data bytes. Q contains 64 text bytes and 24 data bytes. Text base is `0x00400000`, data base `0x10010000`. Sections have four-byte alignment; Q's data additionally requires eight-byte alignment.

1. Give both text starts.
2. Give data starts before the extra alignment requirement.
3. Apply it and give the final exclusive data end.
4. Can the linker arbitrarily choose between two strong global definitions of `value`?

GB-Q04 Recursive frames

Use a simplified sixteen-byte teaching frame, not a full o32 outgoing-argument template. Saved `$ra` is at offset twelve and `$a0` at eight. `sum_to(0)` also allocates a frame.

1. For `sum_to(3)`, find maximum simultaneous frames and bytes.
2. Give saved-state addresses relative to the current `$sp`.
3. Explain unwinding order and why old offsets cannot be reused unchanged after moving `$sp`.
4. Distinguish application ABI preservation from interrupt-context preservation.

GB-Q05 Endianness and alignment

Store `0xdeadbeef` starting at byte address `0x100`.

1. Give four bytes under both byte orders.
2. Analyze `lw` from `0x102` on a strict-alignment machine.
3. In a separate store of halfword `0xbeef` beginning at `0x100`, give bytes and a big-endian `lhu` result.
4. Why does a host memory dump not define a network wire format?

GB-Q06 Exception and syscall state

Model a SPIM-style print-integer service using a conceptual MIPS handler. This is not a claim about SPIM's internal handler implementation. The syscall is not in a delay slot, BD=0, and saved EPC identifies that syscall.

Appendix B Exercises (continued)

1. Identify service, argument, and possible result registers; does print-integer itself have a specified return value?
2. Why can the completed service resume at EPC+4 under these assumptions?
3. Why is that not a general overflow-recovery rule?
4. Explain register preservation and nested-handler risks.

GB-Q07 Pseudo-instructions and I/O

1. Give possible expansions of `move $s0,$s1` and `la $t0, label`.
2. Why does address or PIC mode affect `la`?
3. Give SPIM registers for printing a null-terminated string.
4. Give services and arguments to print an integer and exit, and explain why source-line count does not establish binary size.

GB-Q08 Combined layout and relocation

P.text 0x24 bytes, Q.text 0x14 with 16-byte alignment, text base 0x00401000. P.data 5 bytes, Q.data 4 aligned to eight, data base 0x10010000. Q.foo is text offset 4 and item data offset 0. P.text offset 8 has jal foo; offsets 0x10/0x14 have lui/addiu for item. Use explicit final S+A with addend zero.

1. Find Q section bases and both padding lengths.
2. Find foo, item, jal field/word, and ra.
3. Find high/low immediates; repeat for item 0x1001fffc.
4. Why cannot a cross-region jal be repaired by truncating 26 bits?

GB-Q09 Archives and cyclic references

Use left-to-right extraction to a fixed point per archive without revisiting earlier archives. main references f. libA.A1 defines f/references g, A2 defines h. libB.B1 defines g/references h.

1. Trace members and unresolved sets for main libA libB.
2. Why does listing libA again succeed?
3. If A1/A2 are combined into one member, does the original order still fail? Explain extraction granularity.

Appendix B Exercises (continued)

GB-Q10 Recursion and ABI counterexamples

Use sum_to with entry sp 0x2000, n=2, enough stack, and an allocated base-case frame.

1. Give each sp, saved n/ra address, and return value.
2. Why is moving the post-call load of t0 before jal unsafe? Give a child clobber example.
3. Why is moving the addition into jal's slot wrong?
4. State input/stack preconditions and prove the contract by induction.

GB-Q11 EPC and MMIO control flow

1. Branch 0x00400040 has a slot load 0x00400044 with a recoverable page fault. Find BD/EPC and explain why universal EPC+4 fails.
2. A non-slot syscall 0x00400080 has completed. The handler explicitly chooses next PC. Find its sequential continuation and compare the retry requirement.
3. Receiver control returns 0, 0, 1 and data 0x141. Count control/data reads and final v0. Which device invariant fails if data is read in the beq slot?

GB-Q12 Long branch with delay slots

Use the pre-Release-6 MIPS32 model. `beq $s0,$s1,far` is at 0x00400000 and its delay slot is `nop`; `far=0x00430000`.

1. Compute the original displacement and decide whether it fits a signed 16-bit field.
2. Rewrite it using an inverted branch and `j`, showing both delay slots and the unequal fall-through position.
3. Trace both outcomes, calculate the inverted branch immediate, and state the reachability restriction on `j`.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design, Revised Fourth Edition; Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study